Decomposizione a valori singolari (SVD) per il trattamento di immagini mediche

Autore: Giuseppe Cucchiara

Docenti:
Elisa Francomano, Michele Girfoglio

Università degli Studi di Palermo

Metodi Numerici Avanzati

A.A. 2025-2026

Introduzione

L'SVD è un metodo di applicazione matriciale fondamentale nell'elaborazione delle immagini, poiché consente di:

- **Scomporre** una matrice in componenti principali.
- **Gestire due sottospazi:** quello dei dati (segnale utile) e quello del rumore.
- **Migliorare la qualità:** fondamentale nell'ambito delle immagini mediche per una visione più chiara di strutture anatomiche o anomalie.
- **Comprimere:** rappresentare le informazioni in modo compatto senza perdere dettagli significativi.

Cenni Storici

Lo sviluppo della SVD ha radici profonde nella geometria differenziale:

- **1873 - 1874: E. Beltrami** e **C. Jordan** scoprono indipendentemente i valori singolari come invarianti per forme bilineari.
- **1889: J.J. Sylvester** arriva a risultati simili chiamandoli *"moltiplicatori canonici"*.
- **1910: E. Picard** è il primo a utilizzare il termine **valeurs singulières** e la notazione σ_k.
- **1936: Eckart e Young** forniscono la prima dimostrazione per matrici rettangolari e complesse.
- **Anni '60: Golub e Kahan** introducono l'algoritmo moderno basato sulle trasformazioni di Householder, rendendo il calcolo efficiente.

Definizione di SVD

Sia $M \in \mathbb{R}^{m \times n}$ una matrice di rango r. La SVD consiste nella fattorizzazione:

Equazione Fondamentale

$$M = U \Sigma V^T$$

- U **($m \times m$):** Matrice ortogonale dei vettori singolari sinistri.
- Σ **($m \times n$):** Matrice diagonale con i valori singolari ordinati:

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$$

- V **($n \times n$):** Matrice ortogonale dei vettori singolari destri.

Proprietà: I σ_i sono le radici quadrate degli autovalori di $M^T M$.

Teorema 1 e Decomposizione Spettrale

Sia r il numero di valori singolari positivi tali che:
$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > \sigma_{r+1} = \cdots = 0$.

Proprietà del Rango

Il numero r di valori singolari non nulli corrisponde esattamente al **rango** della matrice $A \in \mathbb{R}^{m \times n}$.

Decomposizione Spettrale: Ogni matrice di rango r può essere scritta come somma di r matrici di rango 1:

$$A = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_r u_r v_r^T \tag{1}$$

- Questa forma è fondamentale per la **compressione**: permette di ricostruire un'approssimazione di A usando solo i primi termini della somma.

SVD vs Diagonalizzazione Classica

La SVD può essere vista come una generalizzazione della diagonalizzazione per matrici arbitrarie (anche non quadrate).

Matrici Quadrate Simmetriche

$$A = Q\Lambda Q^T$$

- Richiede una sola matrice ortogonale Q.

Matrici Arbitrarie (SVD)

$$A = U\Sigma V^T$$

- Richiede due matrici ortogonali (U e V).

L'SVD "diagonalizza" qualsiasi matrice tramite due basi ortonormali distinte.

Criticità del Calcolo Diretto

Il Limite Teorico ($A^T A$)

Sebbene la SVD sia legata agli autovalori di $A^T A$, il calcolo diretto è sconsigliato per matrici di grandi dimensioni:

- **Onerosità computazionale:** Il prodotto $A^T A$ richiede un numero di operazioni eccessivo per dataset medici ad alta risoluzione.
- **Stabilità numerica:** Se A è malcondizionata, il condizionamento peggiora quadraticamente:

$$cond(A^T A) = [cond(A)]^2$$

Algoritmo di Golub-Reinsch

Bidiagonalizzazione di Householder Si riduce la matrice A in forma bidiagonale superiore B mediante una serie di trasformazioni ortogonali $P^{(k)}$ e $H^{(s)}$:

$$B = P^{(n)} \left(\dots \left(P^{(2)} \left(P^{(1)} A H^{(1)} \right) H^{(2)} \right) \dots \right) H^{(n-2)}$$

dove le matrici P e H sono operatori di riflessione di Householder.

Diagonalizzazione della matrice tridiagonale Si applica il metodo QR alla matrice tridiagonale simmetrica $B^T B$. Sapendo che:

$$A^T A = H(B^T B)H^T \quad \text{con} \quad H = H^{(1)} H^{(2)} \dots H^{(n-2)}$$

Attraverso la diagonalizzazione $B^T B = Q \Lambda Q^T$, si ottiene la scomposizione spettrale:

$$A^T A = (HQ)\Lambda(Q^T H^T)$$

Determinazione delle basi ortonormali

- **Vettori singolari destri:** $V = HQ$
- **Vettori singolari sinistri:** calcolati tramite la relazione $u_i = \frac{A v_i}{\sigma_i}$

Interpretazione Geometrica dell'SVD

La trasformazione lineare $x \mapsto Mx$ può essere interpretata come una sequenza di tre trasformazioni geometriche:

- V^T **(Rotazione o Riflessione):** Riunisce i vettori in una nuova base ortonormale nello spazio di partenza $\mathbb{R}^m$.
- Σ **(Scaling):** Ridimensiona ogni coordinata in base ai fattori σ_i (i semiassi dell'iper-ellisse).
- U **(Rotazione o Riflessione):** Effettua la rotazione finale per mappare i vettori nello spazio di arrivo $\mathbb{R}^n$.

Determinante e Orientamento

Il segno di $\det(M)$ determina se le matrici U e V^T effettuano rotazioni pure o includono riflessioni.

Interpretazione Geometrica: La Sequenza

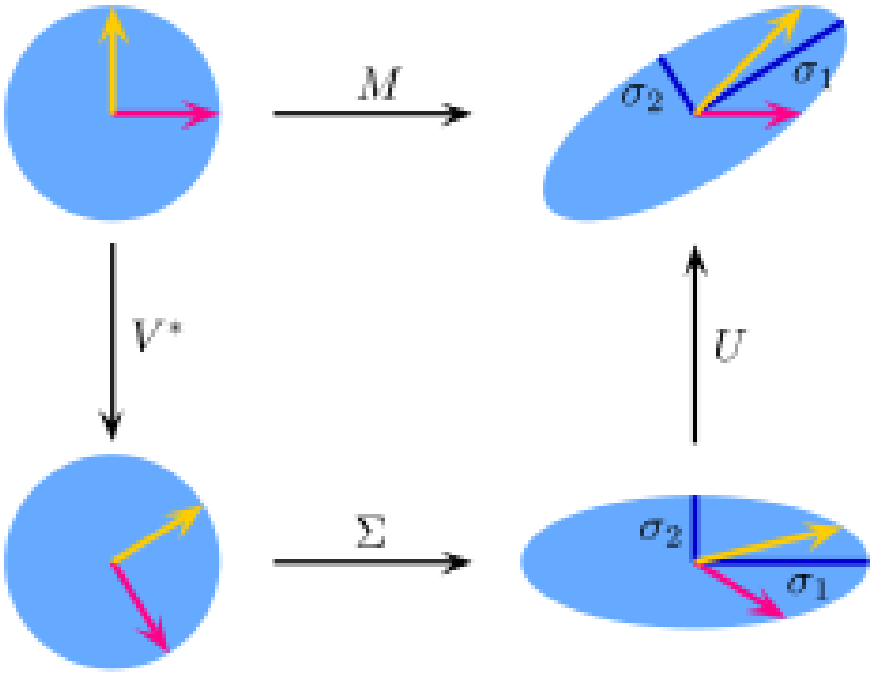
La scomposizione $M = U\Sigma V^T$ scompone qualsiasi trasformazione lineare in tre passaggi geometrici fondamentali:

1. **Rotazione/Riflessione (V^T):** Effettua un cambiamento di base nello spazio di partenza $\mathbb{R}^m$ senza alterare le lunghezze dei vettori.
2. **Ridimensionamento (Σ):** Rappresenta lo *scaling* di ciascuna coordinata. I valori singolari σ_i determinano quanto lo spazio viene "stirato" o "compresso" lungo gli assi.
3. **Rotazione/Riflessione (U):** Effettua la rotazione finale nello spazio di arrivo $\mathbb{R}^n$.

Risultato Visivo

Questa sequenza trasforma una sfera unitaria in un ellissoide i cui semiassi hanno lunghezze pari ai valori singolari σ_i.

Interpretazione Geometrica: Orientamento e Dimensioni



$$M = U \cdot \Sigma \cdot V^*$$

Figura: Trasformazioni SVD

Il ruolo del determinante:

- Se $\det(M) > 0$: possiamo scegliere U e V^T che effetuano rotazioni con riflessione o senza.
- Se $\det(M) < 0$: Si ottiene una rotazione con riflessione e una senza.
- Se $\det(M) = 0$: Il tipo di rotazione può essere scelto indipendentemente.

Caso $m \neq n$: La trasformazione mappa vettori tra spazi di dimensioni diverse ($\mathbb{R}^m \to \mathbb{R}^n$), dove U e V^T agiscono rispettivamente nei due spazi.

Approssimazione Lower Rank (Teorema di Eckart-Young)

Sia $A = U\Sigma V^T$ una matrice di rango r. Fissato un intero $k < r$, definiamo la matrice A_k come:

Definizione di A_k

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_k u_k v_k^T$$

La matrice A_k rappresenta l'approssimazione di rango k della matrice originale A.

- **Proprietà:** Tra tutte le matrici di rango k, A_k è quella che minimizza la distanza da A.
- **Errore di Approssimazione:** L'errore commesso (in norma 2) è pari al primo valore singolare trascurato:

$$\min_{\text{rank}(B)=k} \|A - B\|_2 = \|A - A_k\|_2 = \sigma_{k+1}$$

Importanza per la Compressione e il Rumore

Perché questo teorema è fondamentale per le tue immagini mediche?

1. **Compressione dei Dati:** Invece di memorizzare $m \times n$ pixel, memorizziamo solo i primi k valori singolari e i relativi vettori. Se $k \ll r$, il risparmio di memoria è enorme.
2. **Rimozione Rumore:** Spesso il segnale utile è concentrato nei primi valori singolari $(\sigma_1, \ldots, \sigma_k)$, mentre il rumore è distribuito nei valori più piccoli.

Conclusione Teorica

Troncando la SVD al rango k, "puliamo" l'immagine e ne riduciamo il peso computazionale, mantenendo i dettagli anatomici più rilevanti.

Cos'è la Compressione di un'Immagine?

Comprimere un'immagine significa ricostruirla utilizzando un numero ridotto di informazioni rispetto all'originale.

- Un'immagine digitale è una matrice di pixel con associati valori numerici.
- Tramite la SVD, decomponiamo la matrice e la ricostruiamo usando solo i k valori singolari più significativi.
- **Obiettivo Clinico:** Ottenere un'immagine nitida (es. per individuare una neoplasia maligna) riducendo il peso dei dati.

Implementazione Matlab: SVD ed Energia

Caricamento e Analisi

```
% Caricamento del file DICOM
file = '000007.dcm';

% Lettura dell'immagine
image = dicomread(file);

% Converti l'immagine al tipo double per l'elaborazione con SVD
[U, S, V] = svd(double(image));

% Definizione dei Livelli di Compressione
valori_singolari = [10, 25, 50, 75, 100];

% Valori singolari
sigma = diag(S);

% Energia totale
energia_totale = sum(sigma.^2);

% Preallocazione
informazione_catturata = zeros(size(valori_singolari));

% Calcolo percentuale di informazione
for i = 1:length(valori_singolari)
    K = valori_singolari(i);
    informazione_catturata(i) = ...
        sum(sigma(1:K).^2) / energia_totale * 100;
end
```

Implementazione Matlab: Ricostruzione

Ciclo di visualizzazione Risultati

```
figure;
set(gcf, 'Name', 'Compressione Immagine TC con SVD');

% Aggiungiamo un subplot per l'immagine originale
subplot(2, 3, 1);
% Visualizziamo l'immagine originale
imshow(image, []); title('1. Immagine Originale'); colormap gray;

% Ciclo per ricostruire e visualizzare le immagini compresse
for i = 1:length(valori_singolari)
    K = valori_singolari(i);

    % Riduciamo la dimensione delle matrici per l'approssimazione a basso rango
    U_k = U(:, 1:K);
    S_k = S(1:K, 1:K);
    V_k = V(:, 1:K);

    % Ricostruiamo l'immagine utilizzando la formula dell'approssimazione
    % La matrice risultante e' di tipo double.
    immagine_ricostruita = U_k * S_k * V_k';

    % Aggiungiamo un subplot per l'immagine ricostruita
    subplot(2, 3, i + 1);

    % Visualizziamo l'immagine ricostruita
    imshow(immagine_ricostruita, []); title(['K = ', num2str(K)]); colormap gray;
end
```

Risultati della Compressione

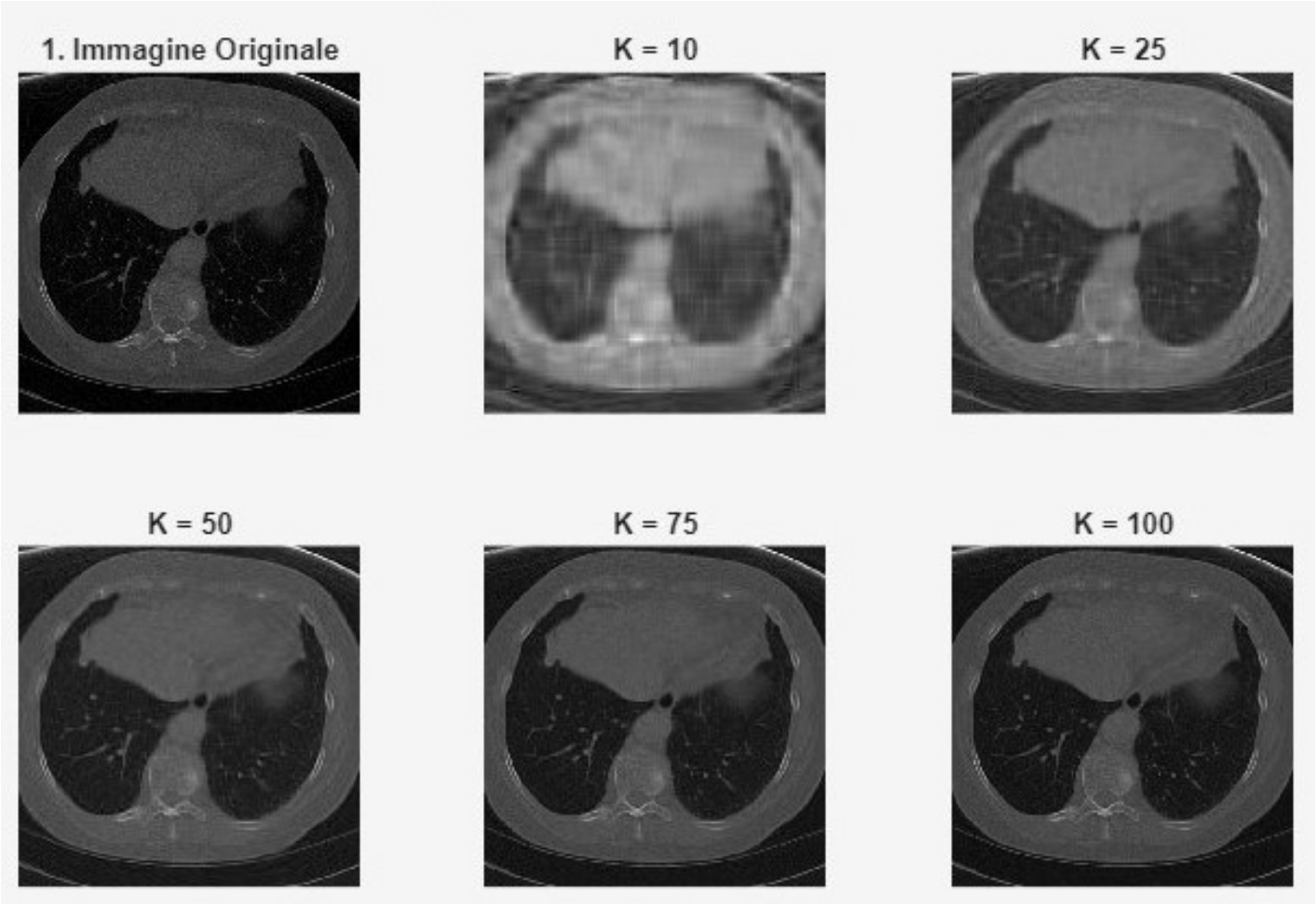


Figura: Evoluzione della qualità dell'immagine TC al variare di K.

Risultati della Compressione

- All'aumentare di K, l'immagine diventa progressivamente più nitida.
- Con $K = 100$, la diagnosi di neoplasia al lobo inferiore è accurata quanto l'originale, ma con meno dati.

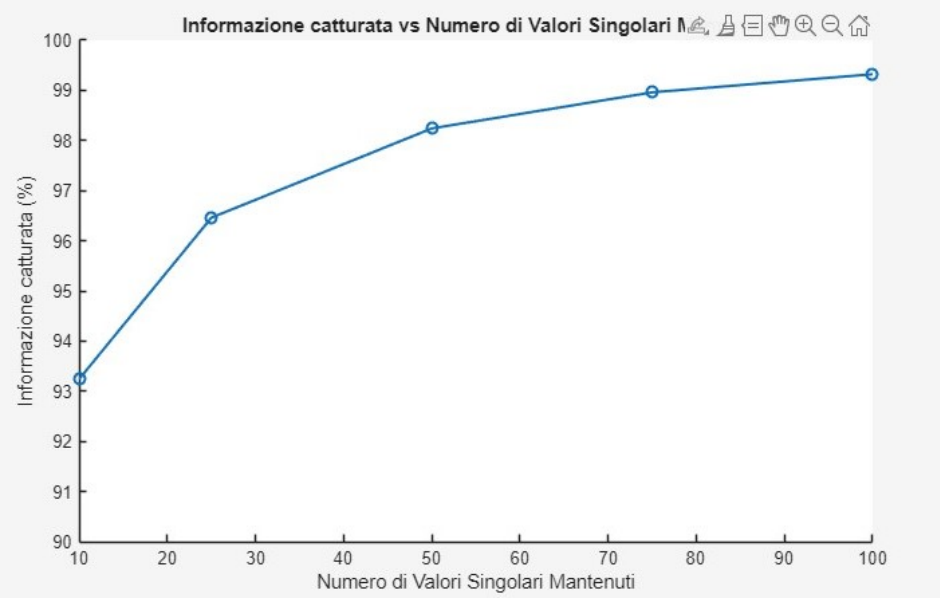
Visualizzazione grafici dell'Informazione Catturata

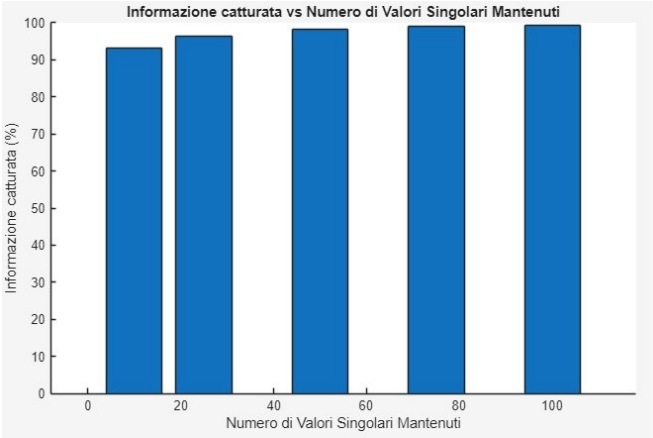
```
figure;
hold on;
plot(valori_singolari, informazione_catturata, '-o', 'LineWidth', 1.5);
xlabel('Numero di Valori Singolari Mantenuti');
ylabel('Informazione catturata (%)');
title('Informazione catturata vs Numero di Valori Singolari Mantenuti');
ylim([90 100]);
hold off;

figure;
hold on;
bar(valori_singolari, informazione_catturata);
xlabel('Numero di Valori Singolari Mantenuti');
ylabel('Informazione catturata (%)');
title('Informazione catturata vs Numero di Valori Singolari Mantenuti');
ylim([0 100]);
hold off;
```

Analisi dell'Informazione Catturata

I grafici confermano l'efficienza della SVD nel trattenere l'energia del segnale:





Nota: Già con pochi valori singolari si supera il 93% dell'informazione totale.

Rapporto di Compressione

Il grado di compressione ottenuto applicando la SVD è espresso dalla formula:

Formula del Compression Ratio (CR)

$$R = \frac{k \cdot n + k + k \cdot m}{m \cdot n} \cdot 100$$

- k: Numero di valori singolari mantenuti.
- m, n: Dimensioni originali dell'immagine.
- R: Indice di compressione espresso in percentuale.

All'aumentare di k, la qualità visiva migliora, ma il rapporto di compressione peggiora (aumenta il peso del file).

Implementazione Matlab: Parte 1

Caricamento e Inizializzazione (Righe 1-13)

```
1  file = '000007.dcm';
2  image = dicomread(file);
3  % Esecuzione SVD
4  [U, S, V] = svd(double(image));
5  m = size(image, 1);
6  n = size(image, 2);
7
8  valori_singolari_da_visualizzare = [10, 20, 50, 100];
9
10 % Inizializzazione corretta
11 rapporto_compressione =
zeros(1,length(valori_singolari_da_visualizzare));
12 rango_di_approssimazione =
zeros(1,length(valori_singolari_da_visualizzare));
13
```

Implementazione Matlab: Parte 2

Ciclo di Troncamento e Visualizzazione (Righe 14-28)

```
14 figure;
15 for i = 1:length(valori_singolari_da_visualizzare)
16     k = valori_singolari_da_visualizzare(i);
17     % Ricostruzione matrice approssimata
18     immagine_compressa = U(:, 1:k)*S(1:k, 1:k)*V(:, 1:k)';
19
20     % Calcolo metrica di memoria (CR)
21     compression_ratio = (((n * k) + k + (m * k))/(m * n))*100;
22     rapporto_compressione(i) = compression_ratio;
23     rango_di_approssimazione(i) = k;
24
25     subplot(2, 2, i);
26     imshow(immagine_compressa, []);
27     title(['k = ' num2str(k) ', CR =
'num2str(compression_ratio) '%']);
28 end
```

Risultati ottenuti con il Troncamento

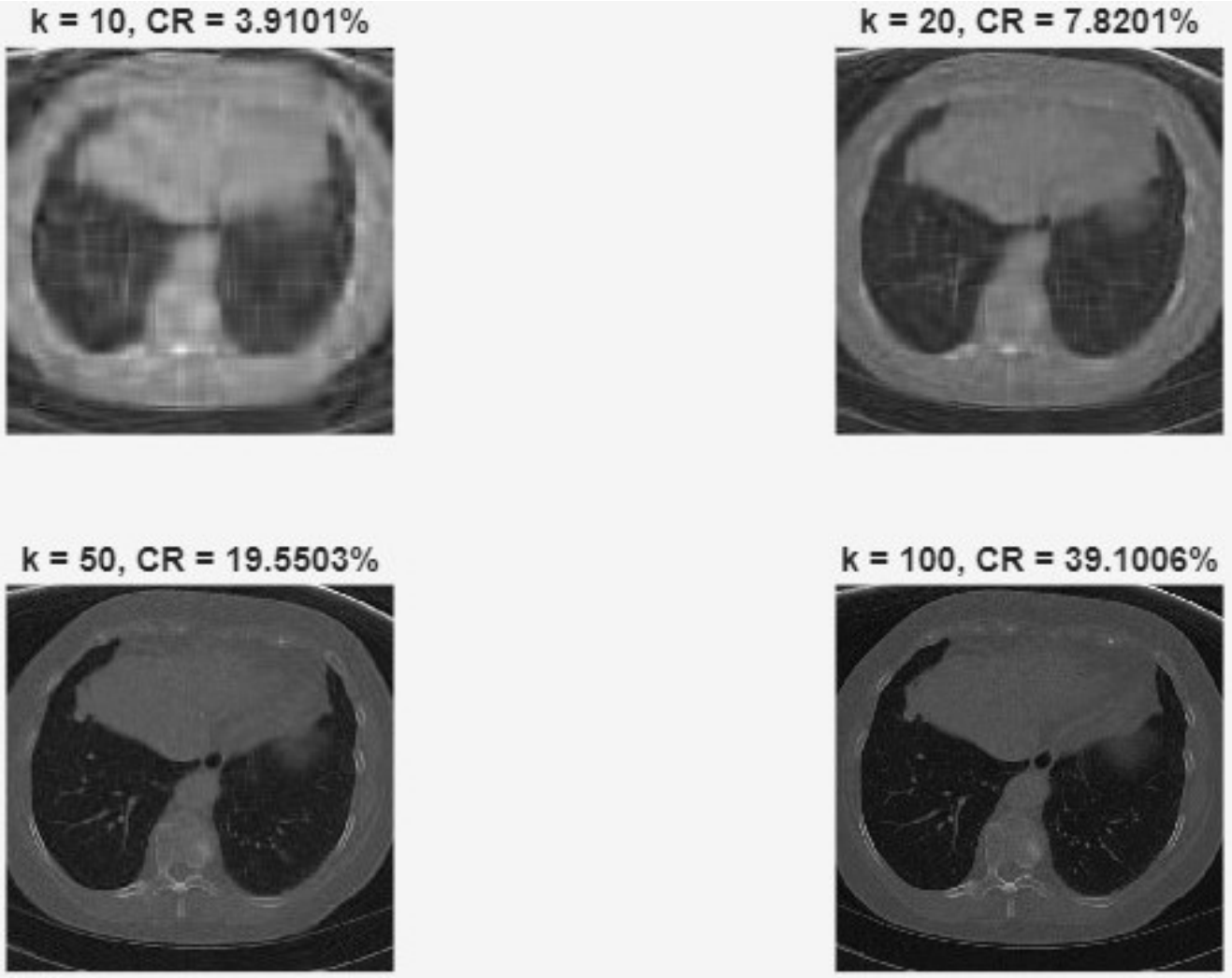


Figura: Visualizzazione delle immagini approssimate.

Analisi del Rapporto di Compressione

Osservando i risultati ottenuti, possiamo trarre le seguenti conclusioni:

- **Flessibilità:** Ogni quadrante mostra il valore di k e il relativo CR calcolato dal codice Matlab.
- **Efficienza:** Notiamo come a $k = 100$ il CR sia circa il 39%, garantendo un'ottima fedeltà diagnostica pur riducendo il peso del file di oltre il 60%.
- **Definizione:** Questa operazione di riduzione dei dati tramite la selezione dei componenti principali è definita matematicamente **Troncamento**.

Osservazione Clinica

Anche con un basso numero di valori singolari, le strutture anatomiche principali restano identificabili, dimostrando la robustezza della SVD.

Gestione del Rumore

Un ruolo molto importante nell'elaborazione delle immagini è la gestione del rumore. Cos'è il rumore? Il rumore è un'alterazione casuale dei pixel che fa apparire l'immagine granulosa, poco nitida e con puntini colorati o grigi.

- In poche parole, è un disturbo visivo dell'immagine compromettendo diverse caratteristiche dell'immagine.
- Per ridurre il rumore è necessario filtrarlo dividendo il nostro spazio vettoriale in due sottospazi distinti.
- Questo permette di migliorare la manipolazione e l'analisi dell'informazione presenti nelle immagini.

Il tutto lo dimostriamo tramite un'implementazione in MatLab:

Implementazione MatLab: Parte 1

Scomposizione e Sottospazio Dati (Righe 1-10)

```
1  file = '000007.dcm'; % File DICOM
2  image = dicomread(file);
3  % Conversione in double
4  [U, S, V] = svd(double(image));
5  % Numero di componenti per il sottospazio dei dati
6  K = 20;
7  % Calcoliamo l'approssimazione a rango K
8  sottospazio_dati_immagine = U(:, 1:K) * S(1:K, 1:K) * V(:, 1:K)';
9  % Calcolo del Rumore Estratto
10 rumore_estratto = double(image) - double(sottospazio_dati_immagine);
```

Implementazione MatLab: Parte 2

Visualizzazione dei Risultati (Righe 11-25)

```
11 figure('Name', 'Riduzione del rumore con SVD');
12 % Visualizzazione dell'Immagine Originale
13 subplot(1, 3, 1);
14 imshow(image, []);
15 title('Immagine Originale');
16
17 % Immagine Filtrata
18 subplot(1, 3, 2);
19 imshow(sottospazio_dati_immagine, []);
20 title(['Immagine Filtrata (K = ' num2str(K) ')']);
21
22 % Rumore Estratto
23 subplot(1, 3, 3);
24 imshow(rumore_estratto, []);
25 title('Rumore Estratto');
```

Analisi dei Risultati

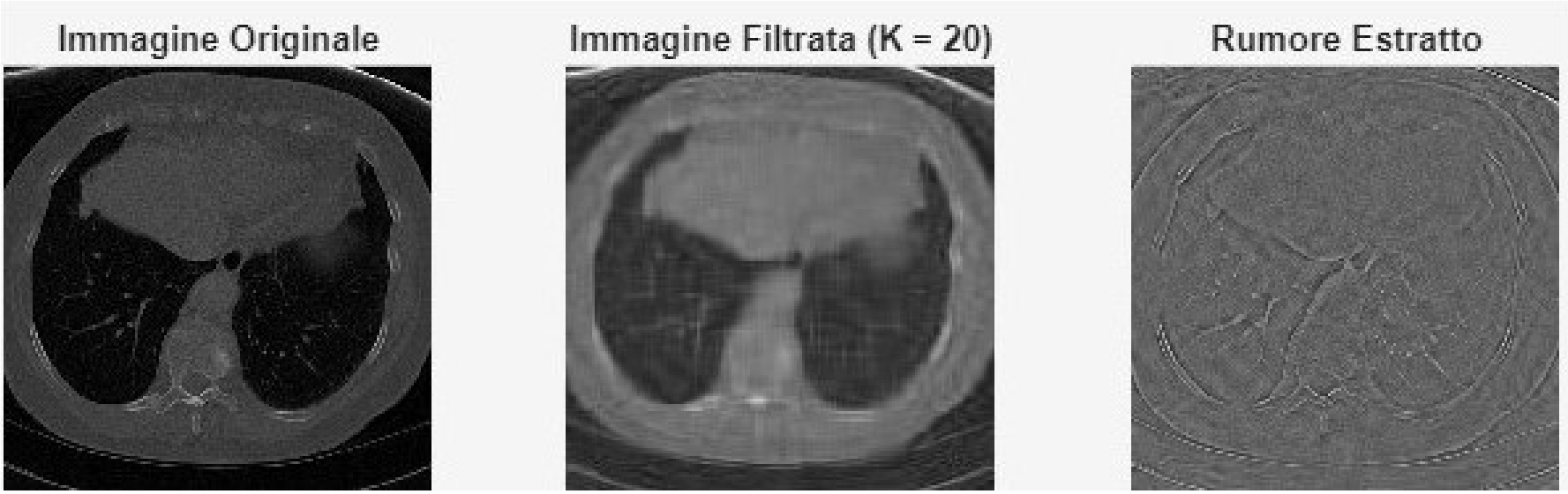


Figura: Risultati dell'implementazione MatLab per il denoising.

Da questa implementazione possiamo notare in Figura che:

- L'immagine originale viene scomposta.
- Le 20 componenti più importanti mantengono l'informazione utile, creando l'immagine filtrata (pulita).
- La parte rimanente scartata diventa il rumore estratto.

Approssimazione Ottimale tramite Troncamento

Il troncamento a rango k è il cuore dell'efficacia della SVD nel trattamento di matrici ad alto rango. Un'immagine è trattata come una matrice A, che viene spesso perturbata dal rumore: $\hat{A} = A + E$.

- **Rumore Gaussiano:** Modello di disturbo stocastico (AWGN) con media zero e distribuzione simmetrica.
- **Natura Casuale:** Ogni pixel nell'immagine $\hat{A}$ è influenzato da un valore casuale, indipendente dagli altri, estratto da questa distribuzione, aggiungendo una quantità di rumore.
- **Effetto:** Produce una "nebbia" o "grana" sottile, rendendo meno definiti i dettagli e introducendo un aspetto leggermente sfocato.
- **Test e Mitigazione:** È spesso usato come modello di base per testare algoritmi di riduzione del rumore.

Compromesso Ottimale

La SVD gestisce il bilanciamento tra:

- **Bias** (errore di approssimazione): L'errore irriducibile $\|A - A_k\|_F$ introdotto dal troncamento forzato della matrice originale A a un rango inferiore k.
- **Varianza** (errore di rumore): Il contributo del rumore E che viene amplificato o mantenuto nel processo.

Recupero del segnale (Denoising):

- Se i valori singolari di A decrescono rapidamente, quelli più piccoli di $\hat{A}$ sono dominati dal rumore E.
- Sotto la condizione $\nu < \sigma_{k+1}(A)$:
  - Il troncamento filtra il rumore in eccesso.
  - L'errore finale è controllato dal segnale scartato $\|A - A_k\|_F$, a meno di un fattore $(1 + O(\epsilon))$.
- Strategia **robusta ed efficiente** anche per matrici ad alto rango.

Implementazione MatLab: Preparazione (Righe 1-12)

Caricamento e Aggiunta Rumore

```
1  % Caricamento e Preparazione dell'Immagine Originale
2  file = 'COVID-101.png';
3  % Carica l'immagine medica e converti in double
4  image = imread(file);
5  A = double(image);
6  [m, n] = size(A);
7  % Aggiunta del Rumore Gaussiano
8  livello_rumore = 30; % Fattore di scala per il rumore
9  rumore = livello_rumore * randn(m, n);
10 A_rumorosa = A + rumore;
11 % Definizione del Rango di Troncamento
12 T = 50;
```

Implementazione MatLab: SVD (Righe 13-24)

SVD e Ricostruzione

```
13 % SVD e Troncamento sulla Matrice Rumorosa
14 [U_hat, S_hat, V_hat] = svd(A_rumorosa);
15 % Troncamento e ricostruzione con solo i primi T componenti
16 U_T = U_hat(:, 1:T);
17 S_T = S_hat(1:T, 1:T);
18 V_T = V_hat(:, 1:T);
19 img_recuperata = U_T * S_T * V_T';
20 % Visualizzazione (subplot)
21 subplot(1, 3, 1); imshow(A, []); title('Immagine Originale');
22 subplot(1, 3, 2); imshow(A_rumorosa, []); title('Immagine Rumorosa');
23 subplot(1, 3, 3); imshow(img_recuperata, []);
24 title(['Immagine Recuperata (T=', num2str(T), ')']);
```

Analisi dei Risultati: Troncamento

Figura: Troncamento con rumore gaussiano.

- L'immagine originale è molto vicina a quella recuperata.
- Differenze notevoli tra immagine recuperata e rumorosa: la SVD ha rimosso la maggior parte del rumore.

Calcolo del Profilo di Intensità (Righe 1-15)

Analisi della 100esima riga

```
1  riga = 100; % Selezione della 100esima riga
2  % Profili di intensità
3  profilo_originale = A(riga, :);
4  profilo_rumoroso = A_rumorosa(riga, :);
5  profilo_recuperato = img_recuperata(riga, :);
6  % Grafico del profilo
7  figure;
8  plot(profilo_originale, 'b', 'LineWidth', 1.5); hold on;
9  plot(profilo_rumoroso, 'r', 'LineWidth', 1);
10 plot(profilo_recuperato, 'g', 'LineWidth', 1.5); hold off;
11 grid on; xlabel('Indice del pixel'); ylabel('Intensità');
12 title("Confronto del Profilo di Intensità");
13 legend('Originale', 'Rumorosa', 'Recuperata');
```

Analisi del Profilo di Intensità

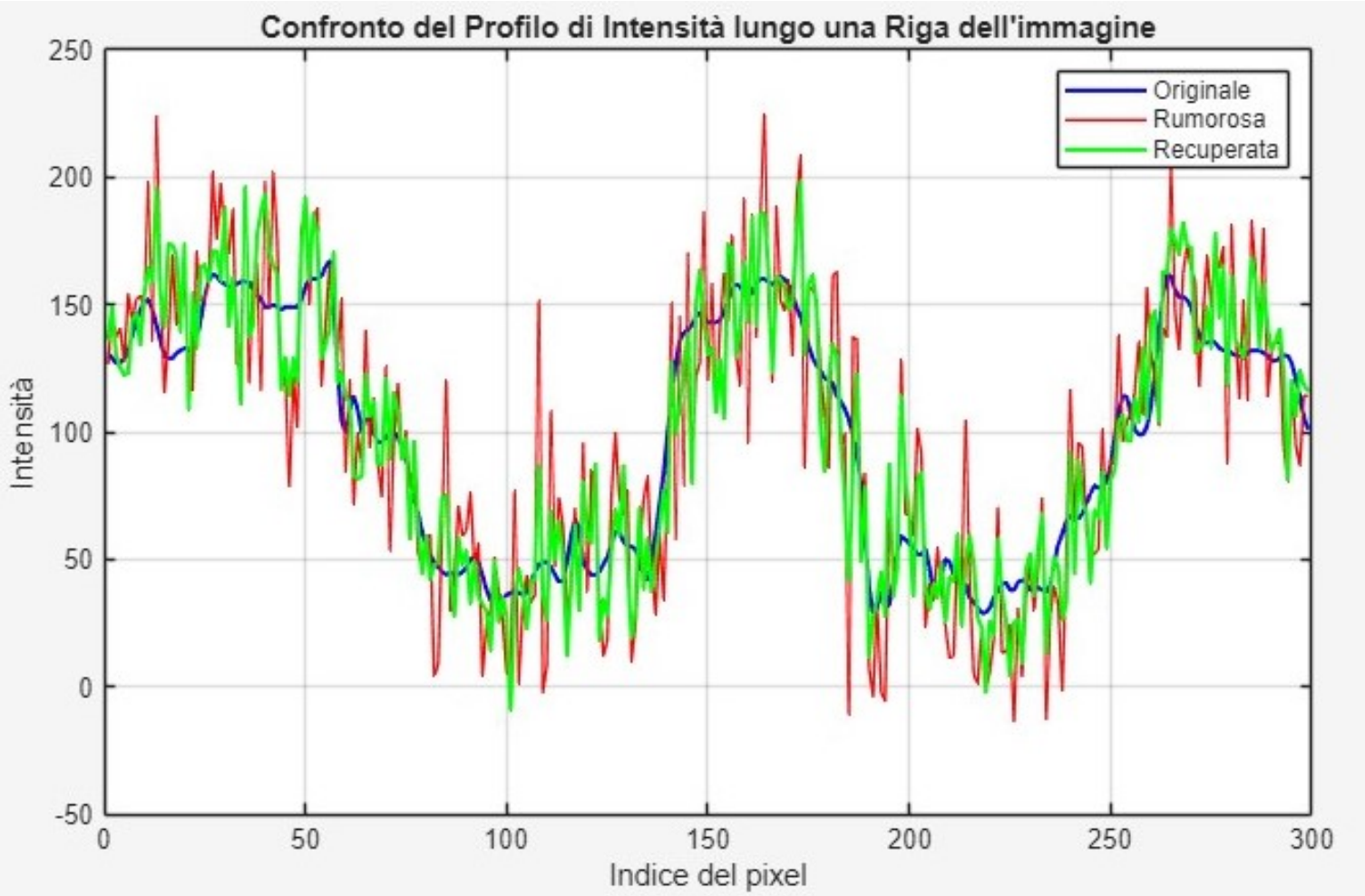


Figura: Grafico del profilo intensità.

Analisi del Profilo di Intensità

- Il rumore gaussiano introduce forti variazioni locali (linea rossa).
- La ricostruzione SVD (linea verde) offre un profilo più regolare e vicino all'originale (linea blu).

Complessità Strutturale

Un fattore determinante per l'efficacia della compressione e del filtraggio tramite SVD è la complessità intrinseca della matrice immagine.

- **Andamento dei Valori Singolari:** Riflette direttamente tale complessità.
- **Struttura Portante:** I primi valori singolari (maggiori) racchiudono la struttura principale dell'immagine.
- **Dettagli e Rumore:** I valori minori rappresentano solitamente dettagli fini o disturbi.
- **Dipendenza dal Contenuto:** Tale andamento non è uniforme, ma dipende strettamente dal contenuto visivo e dalle statistiche del rumore.

Distinzione tra Immagini Lisce e Rugose

Immagini Lisce (Prive di rumore)

I valori singolari presentano una rapida diminuzione (decadimento esponenziale) perché l'informazione è altamente correlata. Ciò comporta un'elevata comprimibilità e concentrazione energetica nei primi termini.

Immagini Rugose (Rumorose)

Il rumore introduce componenti ad alta frequenza che si distribuiscono lungo tutto lo spettro, creando una "coda" di valori non trascurabili.

- **Complessità di Troncamento:** Un k basso comporterebbe perdita di dettagli clinici utili.
- **Dispersione Energetica:** L'energia è dispersa su più componenti, rendendo l'immagine meno adatta alla compressione.

Implementazione MatLab: Preparazione (Parte 1)

Creazione delle Varianti per l'Analisi

```
file = 'COVID-101.png';
% Caricamento e conversione immediata in double
image = double(imread(file));
% Creazione delle due varianti per l'analisi
% Variante "Liscia": applicazione di filtro Gaussiano
% (rimozione alte frequenze)
immagine_liscia = imgaussfilt(image, 2);
% Variante "Rugosa": aggiunta di rumore gaussiano
% (introduzione alte frequenze)
immagine_rugosa = imnoise(uint8(image), 'gaussian', 0, 0.05);
immagine_rugosa = double(immagine_rugosa);
% Calcolo SVD per le due varianti
[U_l, S_l, V_l] = svd(immagine_liscia);
[U_r, S_r, V_r] = svd(immagine_rugosa);
```

Implementazione MatLab: Analisi (Parte 2)

Confronto Visivo e Decadimento

```
% Estrazione valori singolari (vettori diagonali)
s_liscia = diag(S_l);
s_rugosa = diag(S_r);
% Confronto Visivo tra le varianti analizzate
figure('Name', 'Confronto Varianti Immagine');
subplot(1,2,1); imshow(uint8(immagine_liscia)); title('Immagine Liscia');
subplot(1,2,2); imshow(uint8(immagine_rugosa)); title('Immagine Rugosa');
% Decadimento dei Valori Singolari
figure('Name', 'Analisi Valori Singolari');
semilogy(s_liscia(1:100), 'b', 'LineWidth', 2); hold on;
semilogy(s_rugosa(1:100), 'r', 'LineWidth', 2);
title('Analisi Strutturale: Decadimento Valori Singolari');
xlabel('Indice k'); ylabel('Valore Singolare \sigma_k');
legend('Immagine Liscia', 'Immagine Rugosa'); grid on;
```

Analisi dei Risultati

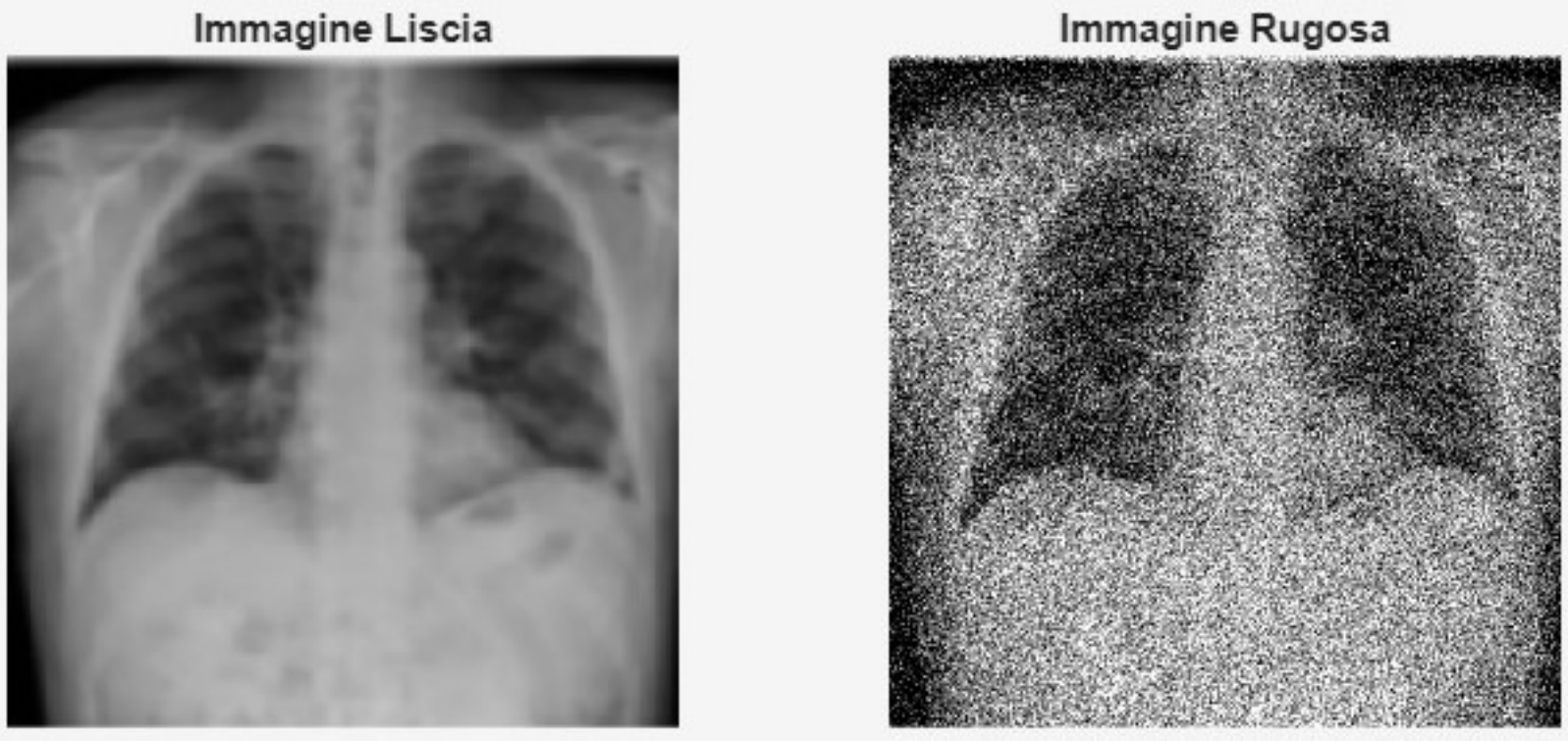


Figura: Differenza tra immagine liscia e rugosa.

L'immagine **liscia** (a sinistra) è ottenuta tramite filtro gaussiano per rimuovere le alte frequenze, mentre l'immagine **rugosa** (a destra) mostra l'effetto del rumore gaussiano aggiunto.

Analisi Strutturale

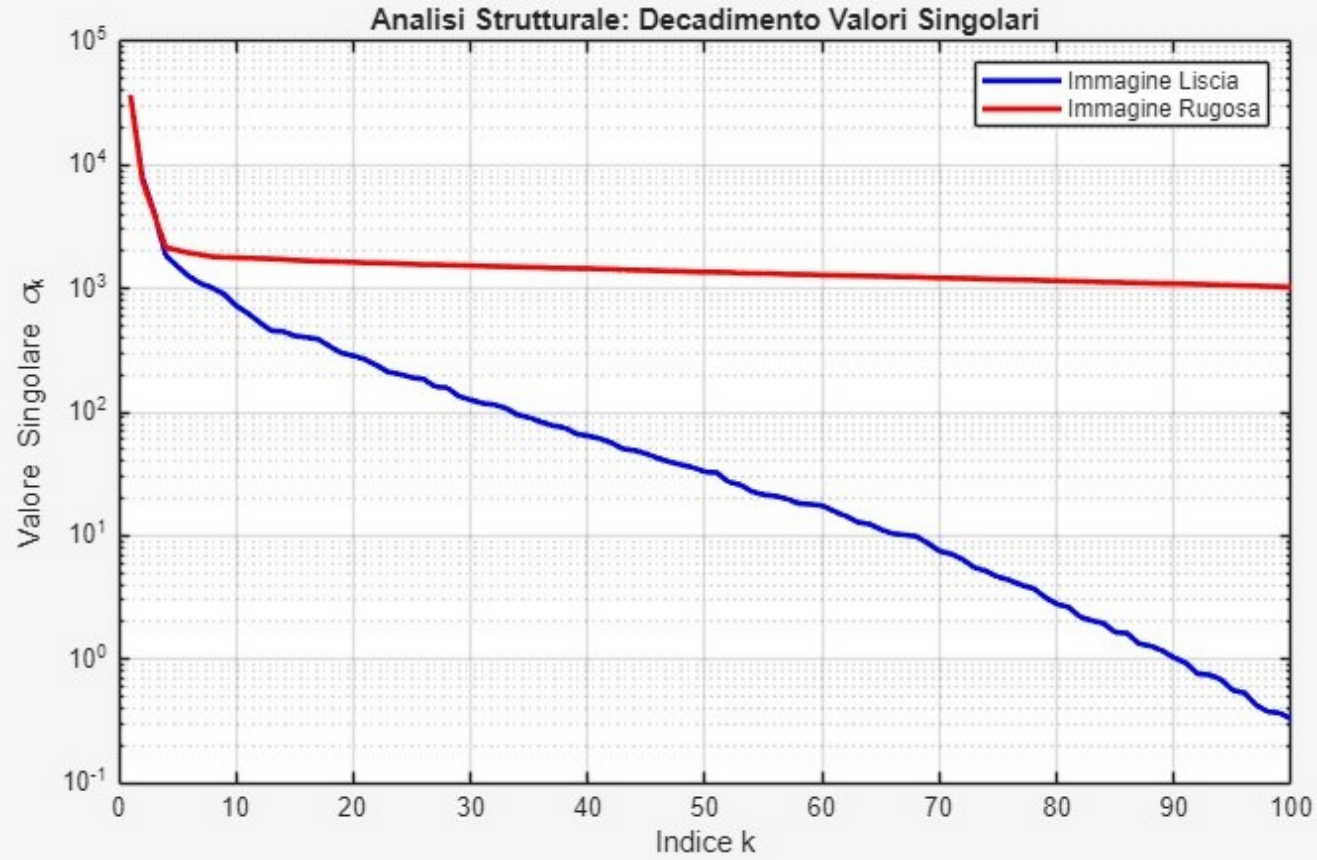


Figura: Decadimento Valori Singolari.

Analisi Strutturale

- **Immagine Liscia:** I valori σ_k decrescono rapidamente verso lo zero, indicando che l'informazione (parenchima e coste) è concentrata in un sottospazio di dimensione $k \ll n$.
- **Immagine Rugosa:** La curva rossa si mantiene costantemente sopra la blu. Questo scarto è la **"spesa energetica"** del rumore stocastico che non può essere compresso efficacemente.

Impatto della Struttura dell'Immagine sulla Ricostruzione

Obiettivo: Valutare l'efficacia dell'approssimazione Low-Rank in base alla complessità strutturale.

- **Metodologia:**
  - Applicazione della SVD alle due varianti: *liscia* e *rugosa*.
  - Fissata una soglia di troncamento uniforme: $k = 50$.
- **Analisi del Test:**
  - Verifica della capacità della SVD di isolare l'informazione anatomica coerente.
  - Osservazione della sensibilità della ricostruzione alla complessità originale.

Ipotesi Sperimentale

La medesima soglia k produrrà risultati qualitativamente diversi: l'immagine più strutturata (rugosa) richiederà potenzialmente un rango maggiore per preservare i dettagli rispetto alla variante liscia.

Implementazione MatLab: Ricostruzione (k=50)

Approssimazione Low-Rank

```
% Scegliamo un rango ridotto
k = 50;
% Ricostruzione dell'immagine liscia
img_liscia_rec = U_l(:,1:k) * S_l(1:k,1:k) * V_l(:,1:k)';
% Ricostruzione dell'immagine rugosa
img_rugosa_rec = U_r(:,1:k) * S_r(1:k,1:k) * V_r(:,1:k)';
% Visualizzazione del confronto delle ricostruzioni
figure('Name', ['Ricostruzione con k = ', num2str(k)]);
subplot(1,2,1); imshow(uint8(img_liscia_rec));
title(['Liscia Ricostruita (k=', num2str(k), ')']);
subplot(1,2,2); imshow(uint8(img_rugosa_rec));
title(['Rugosa Ricostruita (k=', num2str(k), ')']);
```

Impatto Pratico

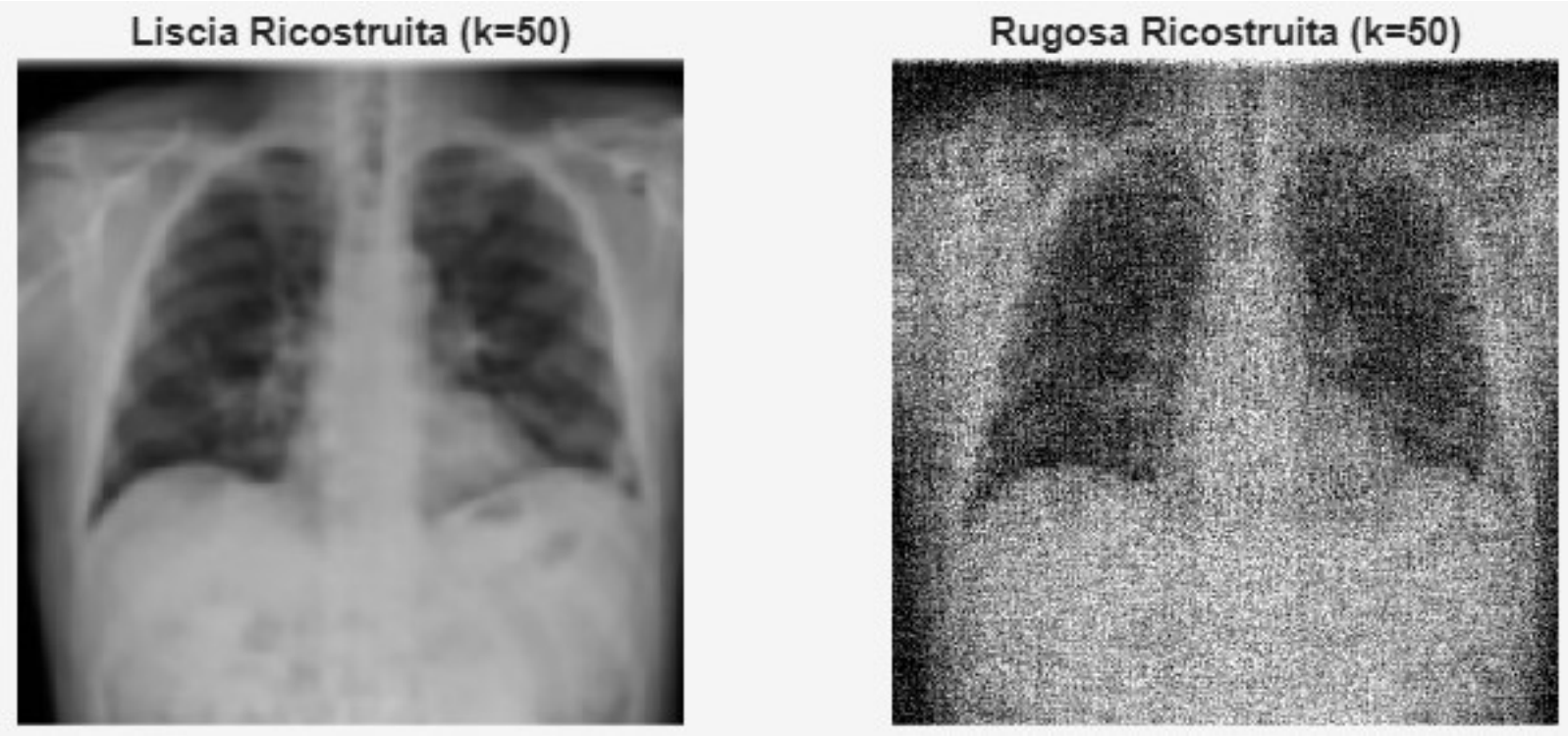


Figura: Ricostruzione immagine liscia e rugosa.

Impatto Pratico

- **Fedeltà strutturale:** L'immagine liscia mantiene un'alta qualità, confermando la concentrazione del segnale nei primi valori.
- **Filtraggio intrinseco:** La variante rugosa subisce una perdita di definizione, ma il troncamento elimina le componenti stocastiche ad alta frequenza scartando la coda dello spettro.

Costo Computazionale

L'SVD effettua tantissime operazioni, specie se vengono utilizzati dataset di grandissime dimensioni, il che ci pone dei limiti nel calcolo a causa dell'elevato costo computazionale.

Analisi del Costo

Data una matrice $m \times n$ e il numero di valori singolari k desiderati:

- **Calcolo SVD completa:** Ha un costo di $O(mn^2)$.
- **Riduzione delle immagini:** Il costo è $O(mk + k^2 + nk)$, poiché manteniamo solo i primi k valori singolari e i relativi vettori di U, S, V.
- **Ricostruzione:** Il costo è analogo a quello della riduzione, basandosi sulle moltiplicazioni tra le matrici troncate.

Costo Totale

Il costo complessivo è $O(mn^2 + mk + k^2 + nk)$. Questo dimostra che l'implementazione è molto costosa computazionalmente.

Importanza del Watermarking

L'implementazione di tecniche di watermarking nelle immagini mediche garantisce l'autenticità, l'integrità e la legittima proprietà dei dati clinici.

- **Massima Robustezza e Stabilità**: I valori singolari (S) riflettono le proprietà algebriche fondamentali dell'immagine. La modifica di questi valori garantisce che l'informazione sia distribuita uniformemente.
- **Alta Impercettibilità**: L'incorporazione di un watermark con un fattore di scala (α) ridotto assicura che il cambiamento visivo sia minimo, preservando l'accuratezza diagnostica.
- **Adattabilità**: La SVD utilizza basi ortogonali non fisse, fornendo una rappresentazione ottimale per qualsiasi blocco di dati.

Implementazione MatLab: Inserimento

Preparazione Dati e Calcolo SVD

```
% Caricamento Immagine DICOM
file = '000007.dcm';
image = dicomread(file);
img = double(image(:,:,1)); % Slice 2D

w_data = dicomread(file);
w = double(w_data(:,:,1));
% Parametri e SVD
alpha = 0.005;
[U, S, V] = svd(img);
w_r = imresize(w, size(S));
% Otteniamo Uw e Vw (la "forma") e Sw (l'energia) del watermark
[Uw, Sw, Vw] = svd(w_r);
% Inserimento coerente:
S_wt = S + alpha * Sw;
w_img = U * S_wt * V';
% Visualizzazione Risultati Inserimento
w_img_u8 = uint8(255 * (w_img - min(w_img(:))) / (max(w_img(:)) - min(w_img(:))));
img_u8 = uint8(255 * (img - min(img(:))) / (max(img(:)) - min(img(:))));
w_u8 = uint8(255 * (w - min(w(:))) / (max(w(:)) - min(w(:))));
figure(1);
subplot(1,3,1); imshow(img_u8); title('Immagine Originale');
subplot(1,3,2); imshow(w_img_u8); title('Immagine con Watermark');
subplot(1,3,3); imshow(w_u8); title('Watermark');
```

Analisi dei Risultati: Invisibilità

Figura: Immagine Originale vs Immagine con Watermark.

- L'occhio clinico non riesce a distinguere l'immagine ospite modificata dall'originale.
- Il risultato è attribuibile al piccolo fattore di scala (α) scelto.

Grafico Inserimento Watermark

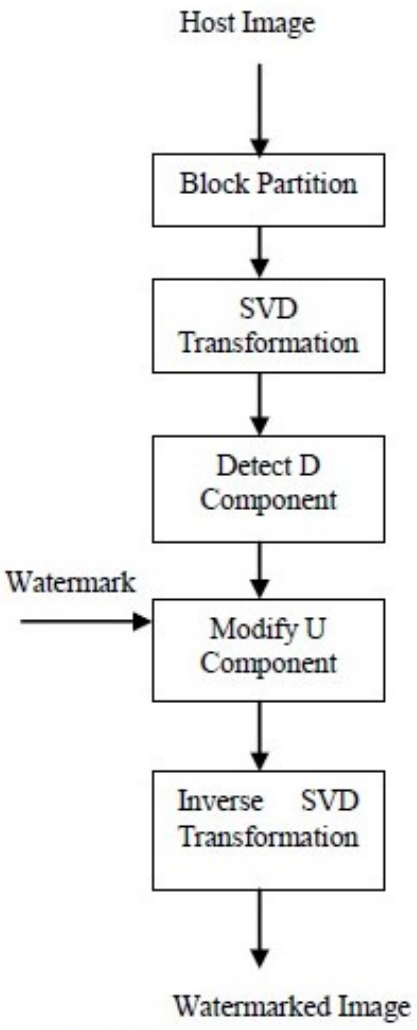


Figura: Struttura grafica dell'inserimento.

Implementazione MatLab: Estrazione

Recupero del Marchio

```
[~, S_w_est, ~] = svd(w_img); % SVD dell'immagine marchiata
Sw_recuperata = (S_w_est - S) / alpha; % Isolo i valori singolari

w_estratto_finale = Uw * Sw_recuperata * Vw';

% Visualizzazione Risultati Estrazione
norm_w_e = uint8(255 * (w_estratto_finale - min(w_estratto_finale(:))) ...
    / (max(w_estratto_finale(:)) - min(w_estratto_finale(:))));

figure(2);
subplot(1,2,1); imshow(w_u8); title('Watermark Originale');
subplot(1,2,2); imshow(norm_w_e); title('Watermark Estratto');
```

Analisi dei Risultati: Robustezza

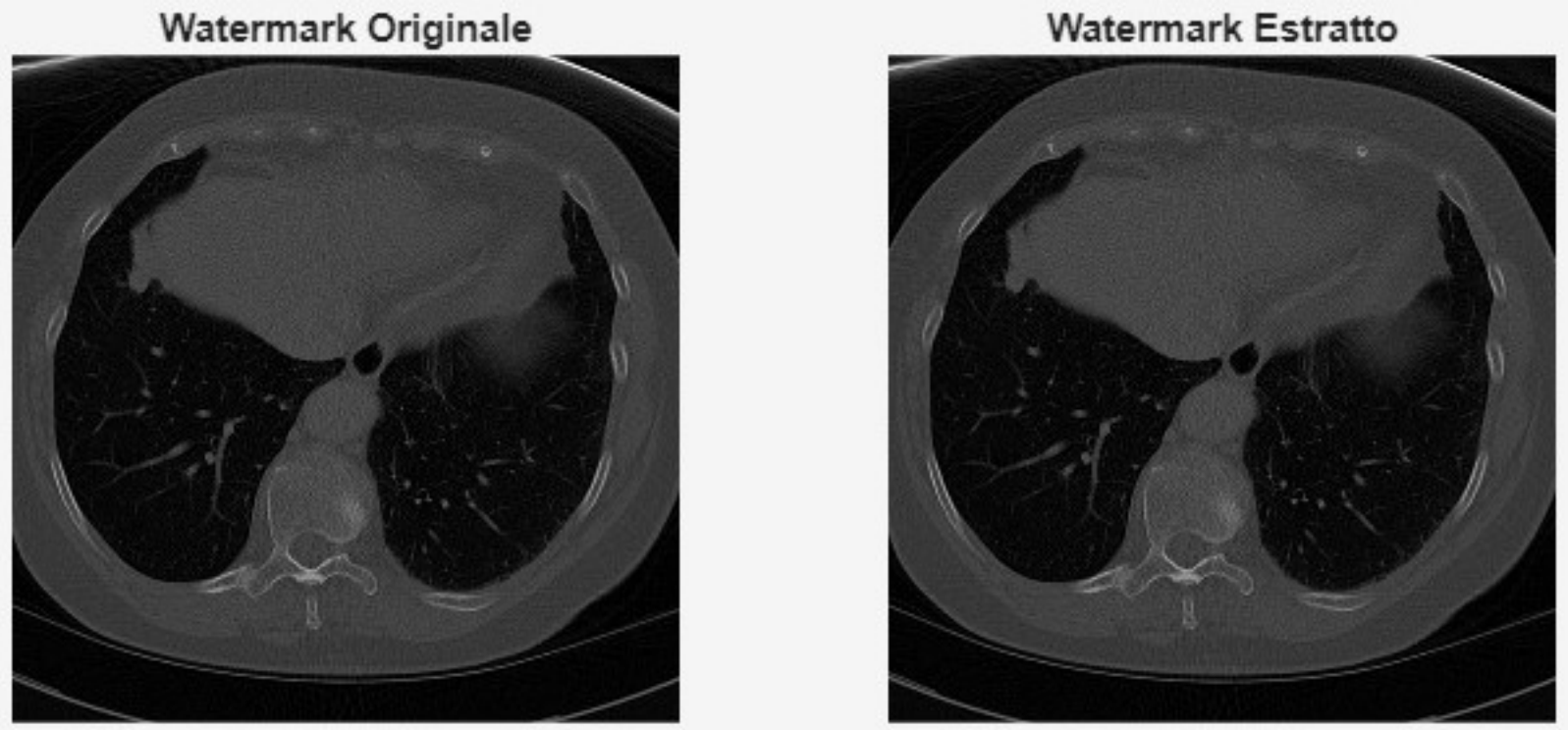


Figura: Watermark Originale vs Watermark Estratto.

- Il watermark estratto è una replica quasi perfetta dell'originale.
- Questo conferma che l'SVD ha protetto efficacemente l'informazione del watermark.

Illustrazione Grafica Watermark Estratto

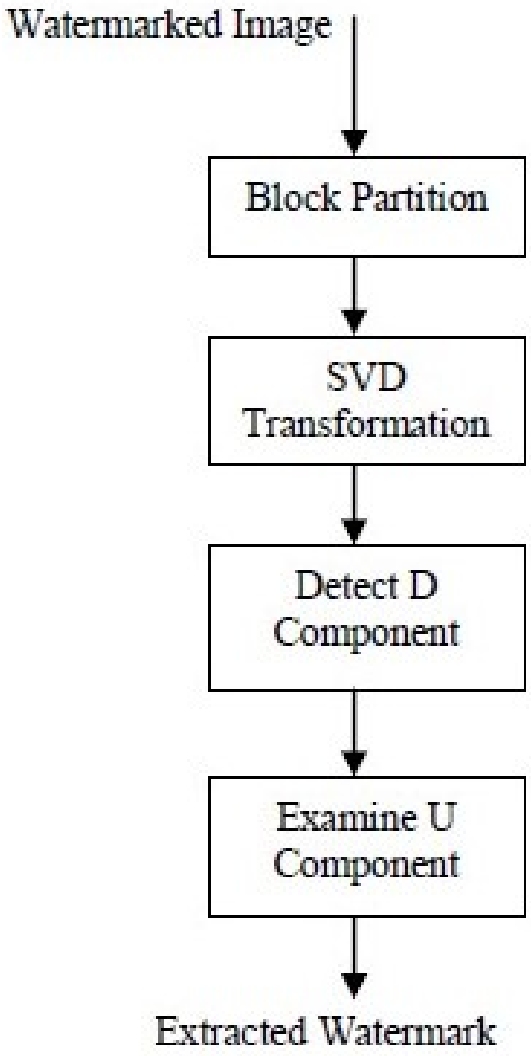


Figura: Struttura grafica dell'estrazione.

Metodi con approssimazione a basso rango

L'SVD completa è un'implementazione molto costosa. Utilizziamo varianti che richiedono un'approssimazione a basso rango considerando il valore singolare più grande che contiene la maggior parte dell'informazione.

Principio della SVD Ridotta

$$M \approx U_k \Sigma_k V_k^T$$

- U_k: matrice contenente solo le prime k colonne di U.
- Σ_k: matrice diagonale con i primi k valori singolari.
- V_k^T: trasposta di una matrice contenente solo le prime k colonne di V.

SVD Sottile

- **Concetto:** Riduce le dimensioni di U e V eliminando le colonne che moltiplicano la parte nulla di Σ.
- **Proprietà:** $M = U_k \Sigma_k V_k^*$ dove $k = \min(m, n)$.
- **Vantaggio:** Ricostruzione **esatta** con minor costo computazionale.

Implementazione MatLab Completa (Righe 1-13)

```
1  file = '000007.dcm';
2  image = dicomread(file);
3  % Calcoliamo la SVD sottile
4  [U, S, V] = svd(double(image), 'econ');
5  % Visualizziamo l'immagine originale
6  subplot(2, 3, 2);
7  imshow(image, []);
8  title('Immagine Originale');
9  % Visualizziamo l'immagine dopo aver applicato SVD sottile
10 immagine_ricostruita = U * S * V';
11 subplot(2, 3, 3);
12 imshow(immagine_ricostruita, []);
13 title('Immagine Ricostruita con SVD sottile');
```

SVD Sottile

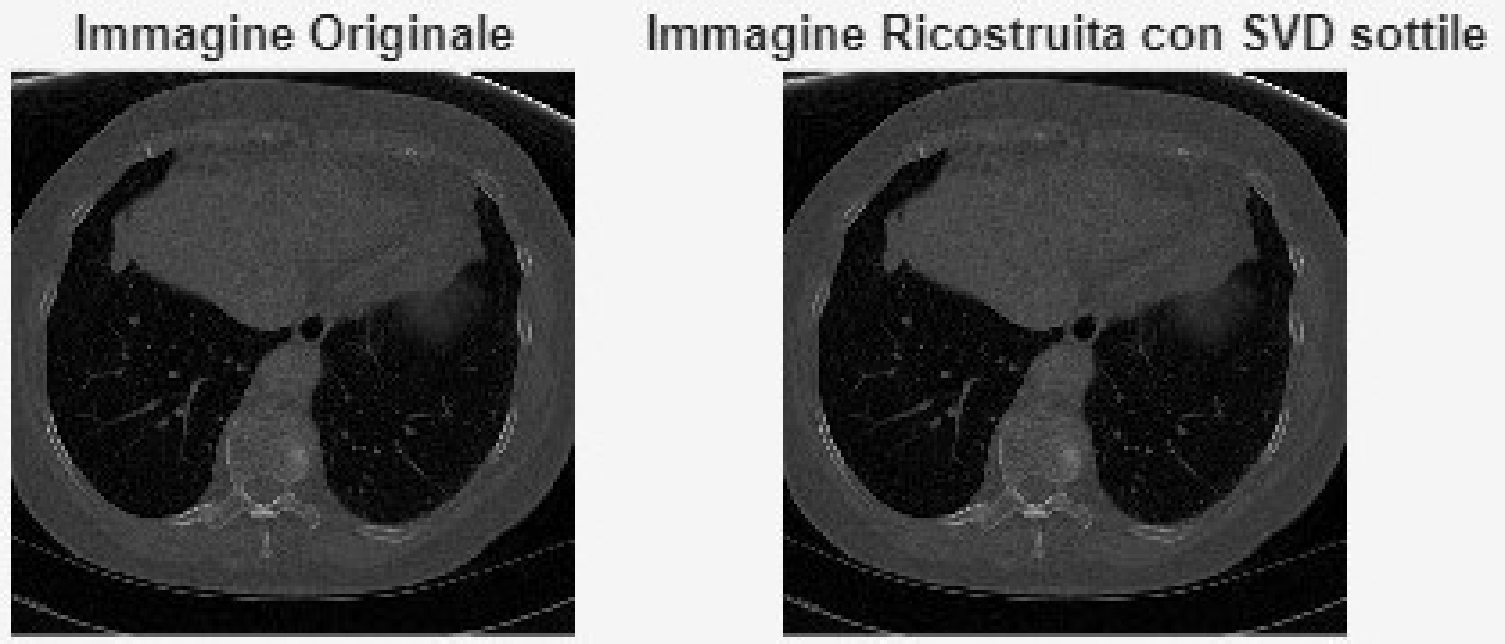


Figura: Implementazione con SVD Sottile.

Il risultato finale di questa implementazione è che otteniamo due foto identiche

SVD Compatta

- **Concetto:** Vengono calcolati solo gli R vettori corrispondenti a valori singolari **non nulli** ($\sigma_i > 0$).
- **Proprietà:** Elimina la ridondanza lineare della matrice.
- **Differenza:** È ancora una ricostruzione **esatta**, ma basata sul rango $R \leq \min(m, n)$.

Implementazione MatLab Completa (Righe 1-18)

```
1  file = '000007.dcm';
2  image = dicomread(file);
3  % Calcoliamo la SVD Compatta
4  [U, S, V] = svd(double(image));
5  % Specifichiamo il numero di valori singolari
6  r = 50;
7  Ur = U(:, 1:r);
8  Sr = S(1:r, 1:r);
9  Vr = V(:, 1:r);
10 % Visualizziamo l'immagine originale
11 subplot(2, 3, 2);
12 imshow(image, []);
13 title('Immagine Originale');
14 % Visualizziamo l'immagine dopo la SVD Compatta
15 immagine_ricostruita = Ur * Sr * Vr';
16 subplot(2, 3, 3);
17 imshow(immagine_ricostruita, []);
18 title(['Immagine Ricostruita con SVD Compatta']);
```

SVD Compatta

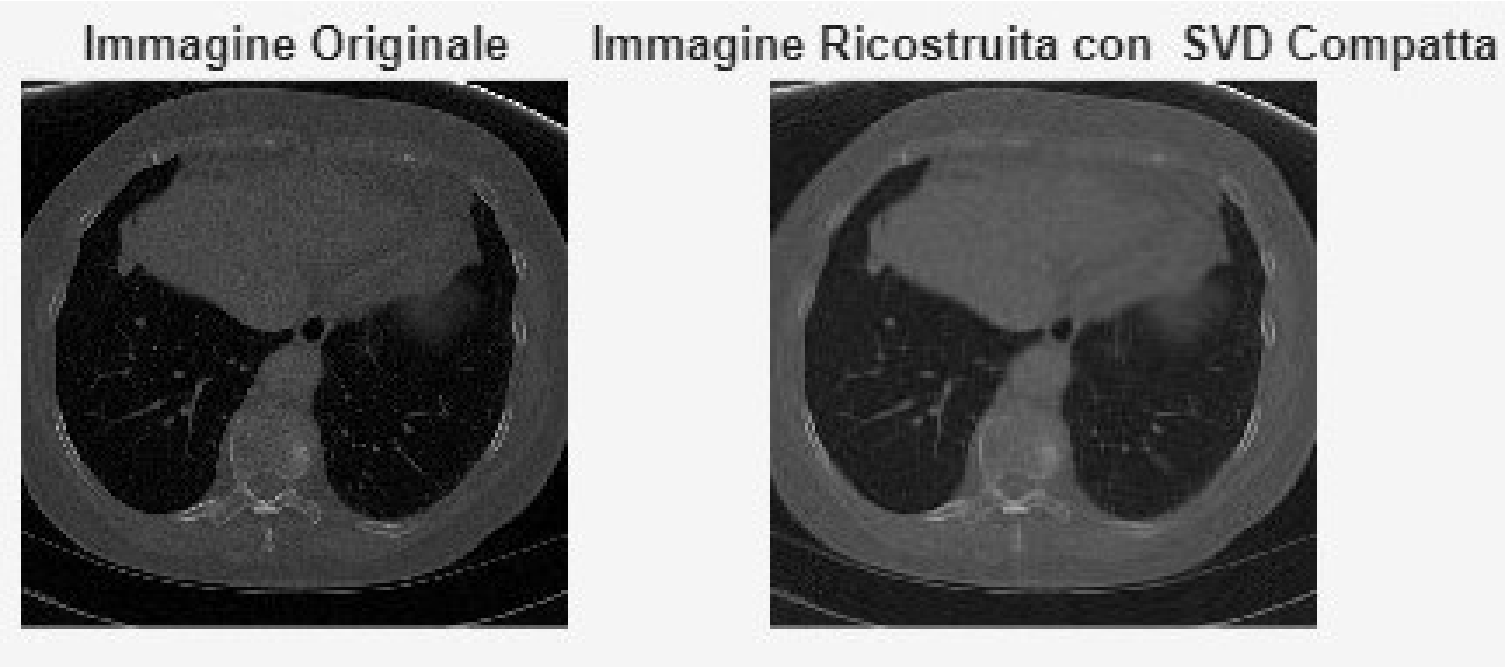


Figura: Implementazione SVD Compatta.

Da questo confronto visivo tra le due immagini, notiamo che quella ricostruita e' la migliore approssimazione dell'originale ottenuta utilizzando solo le 50 componenti principali della SVD.

SVD Troncata

- **Concetto:** Si mantengono solo i primi k valori singolari più grandi, scartando i rimanenti (anche se non nulli).
- **Obiettivo:** Compressione e filtraggio del rumore.
- **Ottimalità:** Fornisce la migliore approssimazione di rango k (Teorema di Eckart-Young).

Implementazione MatLab Completa (Righe 1-19)

```
1  file = '000007.dcm';
2  image = dicomread(file);
3  % Calcoliamo l'SVD Troncata dell'immagine
4  [U, S, V] = svd(double(image));
5  % Numero di valori singolari da mantenere
6  t = 50;
7  Ut = U(:, 1:t);
8  St = S(1:t, 1:t);
9  Vt = V(:, 1:t);
10 % Calcoliamo l'approssimazione dell'immagine originale
11 troncamento = Ut * St * Vt';
12 % Visualizziamo l'immagine originale
13 subplot(2, 3, 2);
14 imshow(image, []);
15 title('Immagine Originale');
16 % Visualizziamo l'immagine dopo SVD Troncata
17 subplot(2, 3, 3);
18 imshow(troncamento, []);
19 title('Immagine Approssimata con SVD Troncata');
```

SVD Troncata

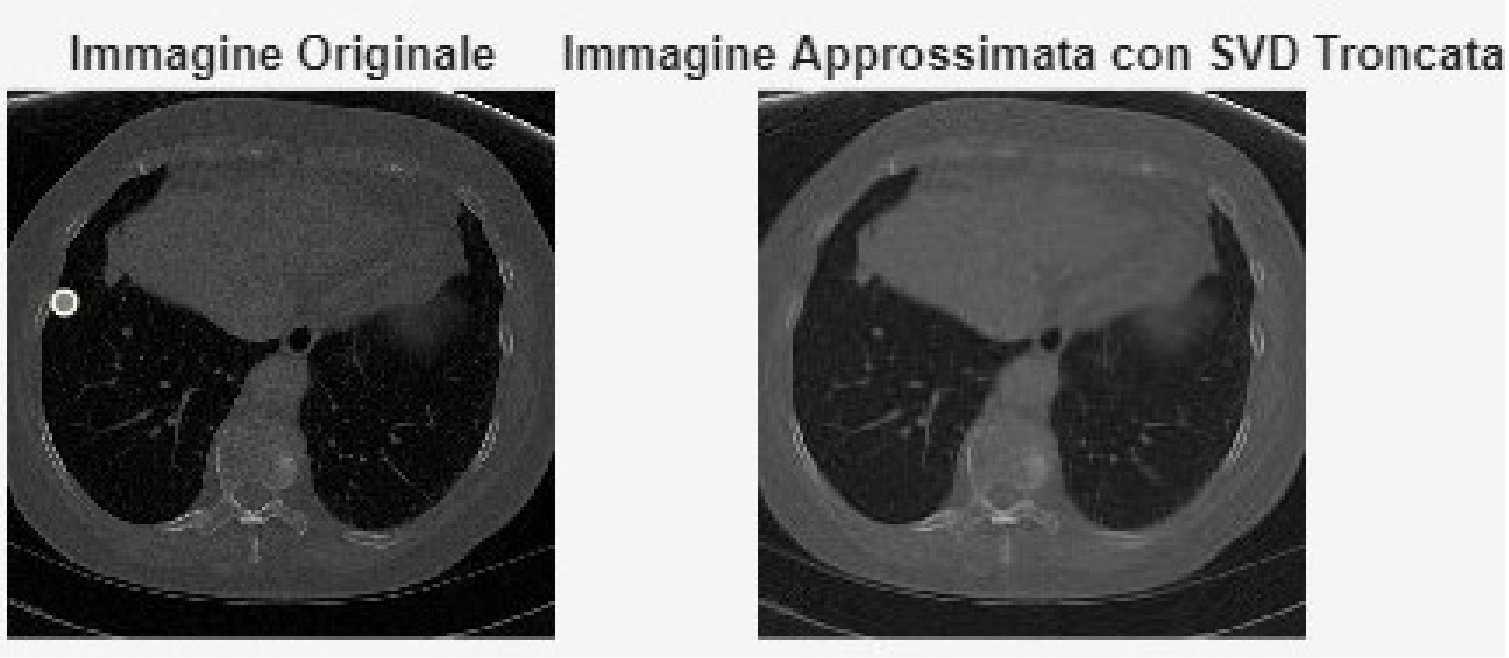
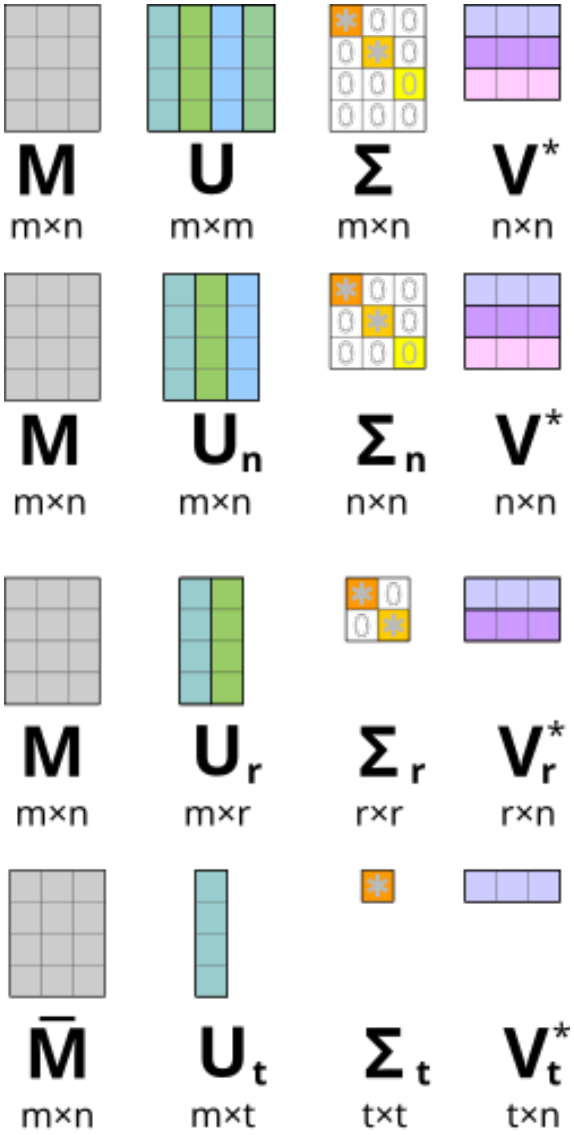


Figura: Implementazione SVD Troncata.

Troncando la SVD ai primi t valori singolari (in questo caso t = 50), si ottenga un'immagine che rappresenta un compromesso tra la compressione dei dati (si memorizzano solo i fattori Ut, St, Vt) e la fedeltà visiva all'originale.

Illustrazione: Confronto tra le SVD



- **SVD Completa**: Decomposizione esatta senza riduzioni.
- **SVD Sottile**: Rimuove le colonne di U non corrispondenti alle righe di V^*.
- **SVD Compatta**: Rimuove i valori singolari nulli e le relative colonne/righe.
- **SVD Troncata**: Mantiene solo i T valori maggiori per l'approssimazione ottimale.

Conclusioni

Lo studio della SVD applicata alle immagini mediche ha evidenziato come questa tecnica sia uno strumento versatile per affrontare le sfide della telemedicina:

- **Efficienza**: L'approssimazione di basso rango consente una compressione dei dati significativa (oltre il 60%) preservando le informazioni cliniche essenziali.
- **Qualità**: Il troncamento dei valori singolari permette un denoising efficace, distinguendo tra il segnale utile e il rumore di fondo.
- **Sicurezza**: L'implementazione del watermarking garantisce l'autenticità e la protezione dei dati sensibili senza alterare l'accuratezza diagnostica.

Riferimenti

- Wikipedia (EN) - Singular Value Decomposition (SVD)
- Dataset per Volumetric DICOM
- COVID-19 Radiography Database
- SVD Based Image Processing Applications
- On the Power of Truncated SVD for General High-rank Matrix Estimation Problems
- A multidimensional image quality measure using Singular Value Decomposition
- SVD based Image Watermarking Scheme